Materia: Programación IV

UNIDAD 3 · PROGRAMACIÓN ORIENTADA A OBJETOS

TRABAJO PRÁCTICO

Integrador POO: de Java a Python

Revisor de la unidad: Prof. Ramiro Hualpa

Puntaje máximo	100 puntos
N.º de TP	
Modalidad	Individual
Entrega	Archivo .zip
Tiempo estimado mínimo	~225 min de desarrollo

Objetivo general

Aplicar las tres inversiones conceptuales de la unidad (compilador → acuerdo, herencia → duck typing, declaración → runtime) y las relaciones estructurales del diagrama de clases sobre un dominio único, produciendo código Python idiomático, no Java traducido.

Marco teórico

Concepto	Aplicación en el proyecto
Encapsulamiento por convención	Python no tiene `private`. El guion bajo simple (`_`) es un acuerdo entre programadores; el doble guion bajo (`__`) es name mangling, no protección.
`@property`	Convierte un atributo en método sin que el código cliente cambie una línea. Solo se justifica cuando hay lógica: validación, cálculo, solo lectura o copia defensiva.
Duck typing	Si tiene el método, sirve. El tipo no se declara: se comprueba en runtime, cuando se usa.
`abc.ABC`	Contrato por herencia explícita. El que lo cumple debe conocerlo y heredar de él. Falla temprano: instanciar sin implementar el `@abstractmethod` revienta al construir.

Concepto	Aplicación en el proyecto
`typing.Protocol`	Contrato estructural. El que lo cumple no necesita conocerlo ni heredar de nada: alcanza con que tenga los métodos. Es el duck typing hecho contrato explícito.
Composición / Agregación / Asociación	Las tres guardan una referencia con la misma sintaxis. Lo que las distingue es quién construye el objeto y quién controla su ciclo de vida.
Copia defensiva	Devolver el contenedor interno rompe el encapsulamiento aunque el atributo sea «privado». Se devuelve una copia.
`@dataclass(frozen=True)`	Objetos de datos inmutables sin escribir cuarenta líneas de asignaciones.
Tabla de equivalencias Java ↔ Python	Mapa que separa lo que se traduce de forma directa de lo que exige volver a pensar el diseño (interfaz, sobrecarga, modificadores de visibilidad).

Caso práctico

Se reutiliza el modelo Figura / Polígono / Lado de la serie de ejercicios, extendido con dos clases nuevas (`Taller` y `Etiqueta`) que fuerzan decisiones de diseño no resueltas en los ejercicios previos.

Diagrama de clases del modelo esperado

El diagrama está escrito en Mermaid, en texto. Para verlo renderizado, se copia el bloque completo y se pega en mermaid.live (no requiere instalar nada ni crear cuenta). También puede re-

producirse en UMLetino o en el editor UML que se venía usando.

```
classDiagram
class Exportable {
    <<Protocol>>
    +exportar() str
}
class Figura {
    <<abstract>>
    #_nombre str
    #_color str
    +area()* float
}
class Poligono {
    <<abstract>>
    #_lados list~Lado~
    +lados_esperados()* int
    +perimetro() float
    +lados() tuple~Lado~
    +exportar() str
}
class Lado {
    #_longitud float
    #_etiqueta Etiqueta
    +longitud float
    +escalar(factor)
}
class Etiqueta {
    <<frozen dataclass>>
    +texto str
}
class Taller {
    #_poligonos list~Poligono~
    +recibir(poligono)
    +restaurar(poligono)
    +inventario() tuple~Poligono~
}
class Triangulo
class Cuadrado
class Pentagono
class Hexagono
class PoligonoRegular {
    <<a revisar en la Parte 3>>
}
class PlanoCAD {
    <<librería externa>>
    +exportar() str
}
Figura <|-- Poligono : herencia
Poligono <|-- Triangulo
Poligono <|-- Cuadrado
Poligono <|-- Pentagono
Poligono <|-- Hexagono
```

```
Poligono "1" *-- "3..*" Lado : composición
Lado "1" --> "0..1" Etiqueta : asociación
Taller "1" o-- "0..*" Poligono : agregación
Poligono ..|> Exportable : cumple
PlanoCAD ..|> Exportable : cumple sin saberlo
```

El diagrama de arriba es el destino, no el punto de partida: se debe entregar una versión propia reflejando las decisiones que se tomen en las Partes 3 y 4 (ver Parte 5). `PoligonoRegular` aparece sin relaciones dibujadas: su lugar en la jerarquía es justamente lo que se pide decidir, así que el diagrama no lo resuelve de antemano.

Material que se entrega

Archivo	Qué es	¿Se modifica?
`parte1_diagnostico.py`	El dominio funcionando, pero escrito con acento de Java.	Sí — es el objeto de trabajo.
`libreria_externa.py`	Clase `PlanoCAD` de un tercero: no hereda de nada propio.	No. Nunca.

Consignas

Ejercicio 1 · Parte 1 — Diagnóstico de java-ismos

20 puntos

~45 min

El módulo `parte1_diagnostico.py` corre de punta a punta sin lanzar un traceback. No tiene bugs de sintaxis: tiene acento.

Contiene exactamente 8 java-ismos de diseño. Ni 7 ni 9: ocho.

El checklist de los siete java-ismos de la Actividad 4 los recorre: están todos acá. El octavo no está en ese checklist. Quien se quede en los 7 que le enseñaron tiene uno de menos, y esa es exactamente la diferencia entre aplicar una lista y haber entendido el criterio.

Además hay ruido sintáctico (punto y coma al final de línea, comparaciones contra `True`, concatenación con `+` donde va un f-string). Ese ruido también hay que limpiarlo, pero no cuenta dentro de los 8.

Se pide

1. Listar los 8 en una tabla dentro de `informe.md`, con este formato: `# | Java-ismo | Dónde (clase.método) | Inversión que lo explica | Síntoma observable`.
2. Corregirlos, justificando cada uno con la inversión conceptual que lo explica. No alcanza con «así es más pythónico»: eso no es una justificación, es una preferencia estética.
3. Al menos 2 de los 8 producen un síntoma reproducible. Para esos dos, se escribe en `demo_sintomas.py` un script corto que demuestre el síntoma antes del arreglo (por ejemplo: `p1._observaciones is p2._observaciones` devolviendo `True`).
4. Para el único getter que sí correspondía convertir en `@property`: se muestra el antes/después probando que el código cliente no cambia una sola línea.

Ejercicio 2 · Parte 2 — Relaciones estructurales

25 puntos

~55 min

Se pide

1. Agregar `Taller`: restaura polígonos. Los recibe ya construidos, no los fabrica.
2. Agregar `Etiqueta`: identifica a un `Lado` con un texto. Se implementa como `@dataclass(frozen=True)`.
3. Implementar ambas mostrando en el constructor la relación que les corresponde:

Relación	Tipo	Multiplicidad
Taller — Poligono	Agregación	0..*
Lado — Etiqueta	Asociación	0..1 (`Etiqueta o None`)
Poligono — Lado	Composición (ya resuelta)	3..*

4. Aplicar copia defensiva en las dos multiplicidades `*`: `Poligono.lados()` y `Taller.inventario()` devuelven una copia, no la lista interna.

La composición Poligono—Lado ya está resuelta en el código de partida: no se reescribe. Se relee con este vocabulario nuevo.

Pregunta obligatoria (va en el informe)

Si la sintaxis de guardar la referencia es idéntica en los tres casos (`self._algo = algo`), ¿cómo se ve en el código la diferencia entre agregación y composición? Se responde para las tres relaciones, señalando la línea exacta que lo delata.

Ejercicio 3 · Parte 3 — Herencia justificada por dominio

20 puntos

~50 min

Se pide

1. Declarar `Poligono` como clase abstracta (`ABC`) con `@abstractmethod lados_esperados()` .
2. Agregar `Pentagono` y `Hexagono` como subclases, más las dos que ya existen. Cada una valida contra `lados_esperados()` .
3. Revisar `PoligonoRegular` . En el código de partida, alguien la modeló heredando de `Poligono` solo para tener el mismo tipo en una lista, es decir, por una necesidad del compilador de Java que en Python no existe.

Decisión obligatoria

Se decide y se justifica cuál de las dos jerarquías se queda y cuál se rediseña, usando el criterio de la unidad: el dominio dice «es-un», frente a la necesidad de un compilador que en Python no existe.

La decisión tiene que quedar implementada en el código, no solo escrita en el informe. Si se decide que `PoligonoRegular` no va por herencia, se muestra con qué se reemplaza.

Falla temprana: instanciar un `Poligono` sin implementar `lados_esperados()` debe reventar al construir, no al usar. Eso se demuestra en el `main.py` (Parte 5).

Ejercicio 4 · Parte 4 — ABC vs. Protocol

15 puntos

~35 min

Se define el contrato `Exportable` con `exportar() -> str`. Deben cumplirlo:

- `Poligono` (dominio propio).
- `PlanoCAD`, la clase de `libreria_externa.py`: ya tiene el método, pero no hereda de nada propio y no se puede modificar.

Se pide

1. Implementarlo como `Protocol`.
2. Escribir una función `exportar_todo(items: list[Exportable]) -> list[str]` que reciba polígonos y planos CAD en la misma lista y funcione en runtime con ambos tipos (contrato estructural / duck typing).
3. Explicar por qué una ABC no hubiera servido para el caso de `PlanoCAD` sin modificarla.

Pregunta que cierra la unidad

La elección entre ABC y Protocol para `Poligono`, ¿la decide el lenguaje o la decide el dominio?

Las Partes 3 y 4 piden la misma decisión desde dos caminos distintos: una llega desde el lenguaje, la otra desde el diagrama. Cuando ambas puedan justificarse con el mismo criterio, la unidad está cerrada.

Ejercicio 5 · Parte 5 — Integradora (entrega final)

20 puntos

~40 min

Con las cuatro partes resueltas, se entrega:

1. Código completo y funcional

`figuras.py` : un solo módulo con el dominio resuelto (Partes 1 a 4). Debe importar `libreria_externa` sin modificarla. Se entrega también `parte1_diagnostico.py` ya corregido, para que se pueda ver el punto de partida y el resultado.

2. Diagrama UML final

`uml/modelo_final.md` : la versión propia del diagrama de clases, reflejando las decisiones de las Partes 3 y 4. Se acepta en cualquiera de estos dos formatos: código Mermaid (texto, en el .md), o una imagen .png exportada desde UMLetino o el editor UML que se use. Debe mostrar herencia, composición (`*--`), agregación (`o--`), asociación (`-->`) y las multiplicidades.

Si el diagrama no coincide con el código, el que vale es el código, y se pierde el punto.

3. Informe (máximo 1 carilla)

`informe.md` debe contener, sí o sí:

- La tabla de los 8 java-ismos (Parte 1).
- La tabla de equivalencias sobre el propio código: entre 5 y 8 filas tomadas de la entrega, con el formato de abajo.
- La respuesta a la pregunta de las tres relaciones (Parte 2).
- La decisión sobre `PoligonoRegular` y su justificación (Parte 3).
- La respuesta a «¿lo decide el lenguaje o el dominio?» (Parte 4).
- El cierre: qué parte del modelo cambió al pasar de Java a Python, y qué parte se mantuvo idéntica.

Formato de la tabla de equivalencias:

Elemento en Java	Cómo quedó en el código Python	¿Traducción directa o rediseño?	Por qué

Ese cierre apunta a distinguir diseño de sintaxis. No se repite la teoría del manual: si la respuesta se puede copiar del capítulo, no responde lo que se pide.

4. Demo ejecutable

`main.py` : un script que arma un taller con al menos 4 polígonos (uno de cada subclase), etiqueta al menos 2 lados, exporta todo junto con un `PlanoCAD` y muestra el inventario por consola. Se aprovecha el demo para dejar a la vista las decisiones de diseño: que un `Lado`

no sobrevive al borrado de su `Poligono` (composición), que un `Poligono` sí sobrevive al del `Taller` (agregación) y que instanciar un `Poligono` abstracto sin `lados_esperados()` revienta al construir (falla temprana).

Conclusiones esperadas

Al finalizar el trabajo práctico, el estudiante debe demostrar:

- **Encapsulamiento por convención**: distinguir el acuerdo (`_`) del name mangling (`__`) y usar `@property` solo donde hay lógica que lo justifique.
- **Traducción con criterio**: separar lo que pasa de Java a Python de forma directa de lo que exige volver a pensar el diseño.
- **Modelado explícito de relaciones**: implementar composición, agregación y asociación distinguiéndolas por ciclo de vida, no por sintaxis.
- **Herencia justificada por dominio**: usar herencia cuando el dominio afirma «es-un», y descartarla cuando era ceremonia del compilador.
- **Contratos estructurales**: elegir entre ABC y Protocol con un criterio argumentable, no por costumbre.
- **Evidencia de las decisiones**: dejar las decisiones de diseño a la vista en el demo ejecutable, no solo afirmadas en el informe.

Modalidad de entrega

Estructura esperada del proyecto

```
Apellido_Nombre_TPI_POO/
├── README.md                # qué resuelve cada archivo
├── figuras.py               # dominio completo (Partes 1 a 4)
├── parte1_diagnostico.py    # el archivo entregado, ya corregido
├── demo_sintomas.py         # demostración de 2 síntomas (Parte 1)
├── libreria_externa.py      # se entrega, SIN MODIFICAR
├── main.py                  # demo ejecutable
├── informe.md               # máximo 1 carilla
└── uml/
    └── modelo_final.md      # diagrama Mermaid (o .png)
```

- Comprimir la carpeta raíz en un archivo .zip.
- Nombre del archivo: `Apellido_Nombre_TPI_POO.zip`.
- Cargarlo en el buzón de entrega, sección «Archivos enviados».
- Antes de comprimir, eliminar `__pycache__/` y `.venv/`.

Verificación previa a la entrega

Antes de subir, se corre el proyecto y se asegura que todo funcione:

```
python main.py           # corre sin errores y muestra el inventario
python demo_sintomas.py  # muestra los 2 síntomas de la Parte 1
```

Recursos adicionales

Documentación oficial

- abc — Abstract Base Classes: https://docs.python.org/es/3/library/abc.html
- typing.Protocol: https://docs.python.org/es/3/library/typing.html#typing.Protocol
- dataclasses: https://docs.python.org/es/3/library/dataclasses.html
- property: https://docs.python.org/es/3/library/functions.html#property
- PEP 8 — Guía de estilo: https://peps.python.org/pep-0008/
- Sintaxis Mermaid para diagramas de clase: https://mermaid.js.org/syntax/classDiagram.html
- Editor Mermaid en línea (pegar y ver el diagrama): https://mermaid.live
- UMLetino (editor UML en línea): https://www.umlet.com/umletino/

Videos y material de la unidad

Todo el material audiovisual de esta consigna está dentro de la Unidad 3, en las Actividades 1 a 8 (Bloques A y B). Para resolver este TP son especialmente relevantes:

- Actividad 1 — Las tres inversiones y la tabla maestra de equivalencias.
- Actividad 2 — La property y el encapsulamiento por convención.
- Actividad 3 — Lo que se traduce mal y las trampas (default mutable, atributo de clase, `super().__init__()`, type hints).
- Actividad 4 — Duck typing, Protocol, dataclass y el checklist de los 7 java-ismos.
- Actividades 5 y 6 — Asociación, agregación y composición (multiplicidades y copia defensiva).
- Actividades 7 y 8 — Clase abstracta, herencia y la elección entre ABC pura y Protocol.